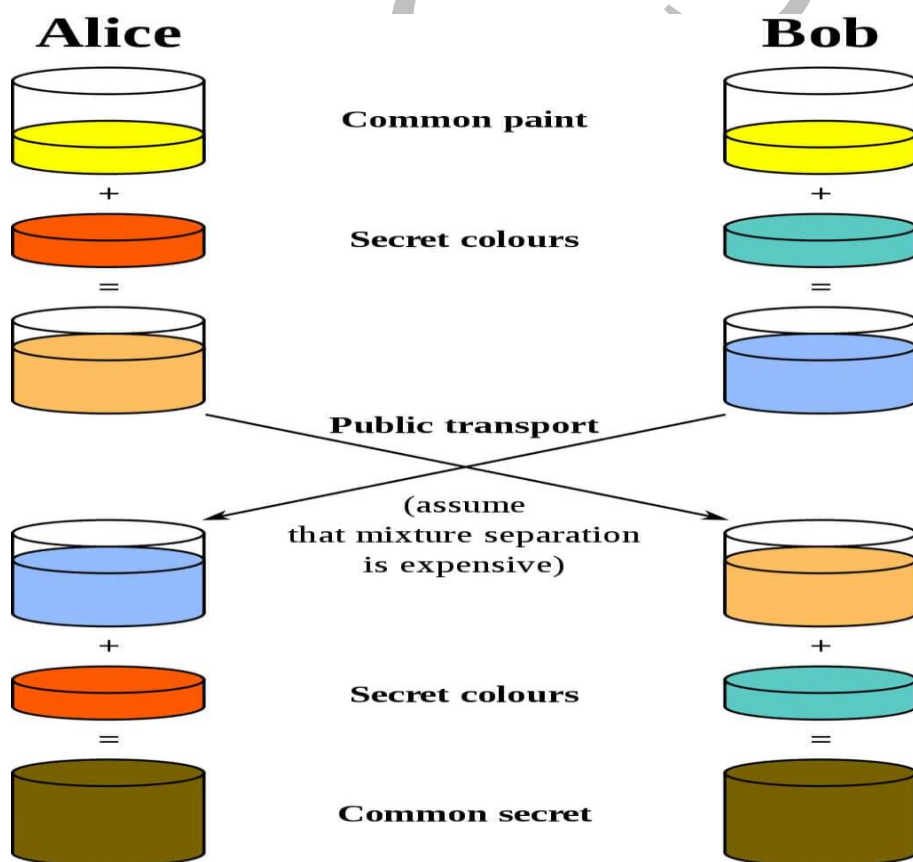


Diffie-Hellman Key Exchange

The Diffie-Hellman key exchange is complex and it can be difficult to get your head around how it works. It uses very large numbers and a lot of math, something that many of us still dread from those long and boring high school lessons.

To make things a bit easier to understand, we will start by explaining the Diffie-Hellman key exchange with an analogy. Once you have a big-picture idea of how it works, we'll move on to a more technical description of the underlying processes.

The best analogy for the Diffie-Hellman scheme is to think of two people mixing paint. Let's use the cryptography standard, and say that their names are Alice and Bob. They both agree on a random color to start with. Let's say that they send each other a message and decide on yellow as their common color, just like in the diagram below:



They set their own color. They do not tell the other party their choice. Let's say that Alice chooses red, while Bob chooses a slightly-greenish blue.

The next step is for both Alice and Bob to mix their secret color (red for Alice, greenish-blue for Bob) with the yellow that they mutually agreed upon. According to the diagram, Alice ends up with an orangish mix, while Bob's result is a deeper blue.

Once they have finished the mixing, they send the result to the other party. Alice receives the deeper blue, while Bob is sent the orange-colored paint.

Once they have received the mixed result from their partner, they then add their secret color to it. Alice takes the deeper blue and adds her secret red paint, while Bob adds his secret greenish-blue to the orange mix he just received.

The result? They both come out with the same color, which in this case is a disgusting brown. It may not be the kind of color that you would want to paint your living room with, but it is a shared color nonetheless. This shared color is referred to as the common secret.

The critical part of the Diffie-Hellman key exchange is that both parties end up with the same result, without ever needing to send the entirety of the common secret across the communication channel. Choosing a common color, their own secret colors, exchanging the mix and then adding their own color once more, gives both parties a way to arrive at the same common secret without ever having to send across the whole thing.

If an attacker is listening to the exchange, all that they can access is the common yellow color that Alice and Bob start with, as well as the mixed colors that are exchanged. Since this is really done with enormous numbers instead of paint, these pieces of information aren't enough for the attack to discern either of the initial secret colors, or the common secret (technically it is possible to compute the common secret from this information, but in a secure implementation of the Diffie-Hellman key

exchange, it would take an unfeasible amount of time and computational resources to do so).

This structure of the Diffie-Hellman key exchange is what makes it so useful. It allows the two parties to communicate over a potentially dangerous connection and still come up with a shared secret that they can use to make encryption keys for their future communications. It doesn't matter if any attackers are listening in, because the complete shared secret is never sent over the connection.

The technical details of the Diffie-Hellman key exchange

Time for some math...

Don't worry, we'll take it slow and try to make the whole process as easy to understand as possible. It follows a similar premise as the analogy shown above, but instead of mixing and sending colors, the Diffie-Hellman scheme actually makes calculations based on exceptionally-large prime numbers, then sends them across.

To ensure security, it is recommended that the **prime (p) is at least 2048 bits long**, which is the binary equivalent of a decimal number of about this size:

415368757628736598425938247569843765827634879128375827365
928736
842736847289385729837592834759348759384759284759287395872
49587
298739587298357928759827958375298763482736857298435793487
95827
938579287395487723975928375924785938670459867923847378267
35267
354762356873486938694567345682765949806384902487580960394
7902
794598273018743975928462095029375928704950293805892098394
5872
094860298491283750294801937109248019358103799581093750193
8507913

957109375970193850891039510730587103937019347019380918039
84091804
981093801985013984019835091835019830910791803958103951903
95180935
8109385019840193580193840198340918093851098309180019

To prevent anyone's head from exploding, we'll run this explanation through with much smaller numbers. Be aware that the Diffie-Hellman key exchange would be insecure if it used numbers as small as those in our example. We are only using such small numbers to demonstrate the concept in a simpler manner.

In the most basic form of the Diffie-Hellman key exchange, Alice and Bob begin by mutually deciding upon two numbers to start with, as opposed to the single common point in the example above. These are the modulus (p) and the base (g).

In practical use, the modulus (p) is a very large prime number, while the base (g) is relatively small to simplify calculations. The base (g) is derived from a cyclic group (G) that is normally generated well before the other steps take place.

For our example, let's say that the modulus (p) is **17**, while the base (g) is **4**.

Once they have mutually decided on these numbers, Alice settles on a secret number (**a**) for herself, while Bob chooses his own secret number (**b**). Let's say that they choose:

$$a = 3$$

$$b = 6$$

Alice then performs the following calculation to give her the number that she will send to Bob:

$$A = g^a \bmod p$$

In the above calculation, **mod** signifies a modulo operation. These are essentially calculations to figure out the remainder after dividing the left side by the right. As an example:

$$15 \bmod 4 = 3$$

If you understand how modulo operations work, you can do them yourself in the following calculations, otherwise you can use an online calculator.

So let's put our numbers into the formula:

$$A = 4^3 \bmod 17$$

$$A = 64 \bmod 17$$

$$A = 13$$

When we do the same for Bob, we get:

$$B = 4^6 \bmod 17$$

$$B = 4096 \bmod 17$$

$$B = 16$$

Alice then sends her result (A) to Bob, while Bob sends his figure (B) to Alice. Alice then calculates the shared secret (s) using the number she received from Bob (B) and her secret number (a), using the following formula:

$$s = B^a \bmod p$$

$$s = 16^3 \bmod 17$$

$$s = 4,096 \bmod 17$$

$$s = 16$$

Bob then performs what is essentially the same calculation, but with the number that Alice sent him (A), as well as his own secret number (b):

$$s = A^b \bmod p$$

$$s = 13^6 \bmod 17$$

$$s = 4,826,809 \bmod 17$$

$$s = 16$$

As you can see, both parties ended up with the same result for s , **16**. This is the shared secret, which only Alice and Bob know. They can then use this to set up a key for symmetric encryption, allowing them to safely send information between themselves in a way that only they can access it.

Note that although B and s are the same in the example above, this is just a coincidence based on the small numbers that were chosen for this illustration. Normally, these values would not be the same in a real implementation of the Diffie-Hellman key exchange.

Even though much of the above data is sent across the channel in cleartext (p , g , A and B) and can be read by potential attackers, the shared secret (s) is never transmitted. It would not be practical for an attacker to calculate the shared secret (s) or either of the secret numbers (a and b) from the information that is sent in cleartext.

Of course, this assumes that the Diffie-Hellman key exchange is properly implemented and sufficiently large numbers are used. As long as these provisions are adhered to, the Diffie-Hellman key exchange is considered a safe way to establish a shared secret which can be used to secure future communications.

Establishing a shared key between multiple parties

The Diffie-Hellman key exchange can also be used to set up a shared key with a greater number of participants. It works in the same manner,

except further rounds of the calculations are needed for each party to add in their secret number and end up with the same shared secret.

Just like in the two-party version of the Diffie-Hellman key exchange, some parts of the information are sent across insecure channels, but not enough for an attacker to be able to compute the shared secret.

Why is the Diffie-Hellman key exchange secure?

On a mathematical level, the Diffie-Hellman key exchange relies on one-way functions as the basis for its security. These are calculations which are simple to do one way, but much more difficult to calculate in reverse.

More specifically, it relies on the Diffie-Hellman problem, which assumes that under the right parameters, it is infeasible to calculate gab from the separate values of g , ga and gb . There is currently no publicly known way to easily find gab from the other values, which is why the Diffie-Hellman key exchange is considered secure, despite the fact that attackers can intercept the values p , g , A , and B .

Attacks against encryption

1. Brute force attacks

Brute-force attacks involve trying every possible character combination to find the 'key' to decrypt an encrypted message. While brute-force attacks may take a smaller amount of time for smaller key spaces, it will take an immeasurable amount of time for larger keyspaces. Hence it is impractical to try brute-force attacks modern encryption systems.

2. Cipher-only attack

In the 'cipher-only' attack, the attacker knows the ciphertext of various messages which have been encrypted using the same encryption algorithm.

The attacker's challenge is to figure the 'key' which can then be used to decrypt all messages.

The 'cipher-only' attack is probably one of the easiest attacks to commit since it is easy to capture the ciphertext (by sniffing) but difficult to implement since the knowledge about the encryption process is limited.

3. Known-plaintext attack

In the 'known-plaintext' attack, the attacker knows some of the plaintext and the ciphertext. He then has to figure the 'key' by reverse engineering and he can decipher other messages which use the same 'key' and algorithm.

The 'known-plaintext' attack was effective against simple ciphers such as the 'substitution cipher'. It was popular for breaking ciphers used during the Second World War.

4. Chosen plaintext attack

The 'chosen-plaintext' attack is similar to the 'known-plaintext' attack, but here the attacker experiments by choosing his own plaintext (say choosing a word such as 'cryptography') for a 'Vignere cipher' and with the generated ciphertext he can figure the 'key'.

Once he figures the 'key' he can learn more about the whole encryption process and understand how the 'key' is being used.

With this information, he can decrypt other messages.

5. Chosen ciphertext attack

In the 'chosen ciphertext' attack, the attacker chooses a portion of the decrypted ciphertext. He then compares the decrypted ciphertext with the plaintext and figures out the key.

This is relatively a harder type of attack and earlier versions of RSA were subject to these types of attacks.

6. Differential cryptanalysis

This was a popular type of attack against block algorithms such as DES in 1990. The primary aim of this attack, as with other attacks, is finding the 'key'.

The attacker follows several messages of plaintext into their transformed ciphertext. He observes the changes from plaintext to the ciphertext and deduces the key.

This is a type of ‘chosen-plaintext’ attack since the attacker chooses the plaintext to observe the transformation.

7. Linear cryptanalysis

The attacker carries out a “known-plaintext” attack against several messages which have been encrypted with the same key. This gives the attacker insight into the probability of a particular key. If more messages are attacked, there is a higher possibility of finding the particular “key”.

8. Side channel attacks

Apart from just relying on mathematical ways to break into systems, attackers may use other techniques such as observing power consumption, radiation emissions and time for data processing. With this data, the attacker works in a reverse manner to figure the ‘keys’ to an algorithm just by observing the amount of heat released in an attack.

RSA keys in 1995 were uncovered using this type of attack.

9. Replay attacks

In a ‘replay attack’, the attacker captures some information (as an example, authentication information) and re-submits it back to the receiver. This dupes the receiver and they give the attacker unauthorized access.

Timestamps are one of the important countermeasures to handle ‘replay attacks.’

HMAC

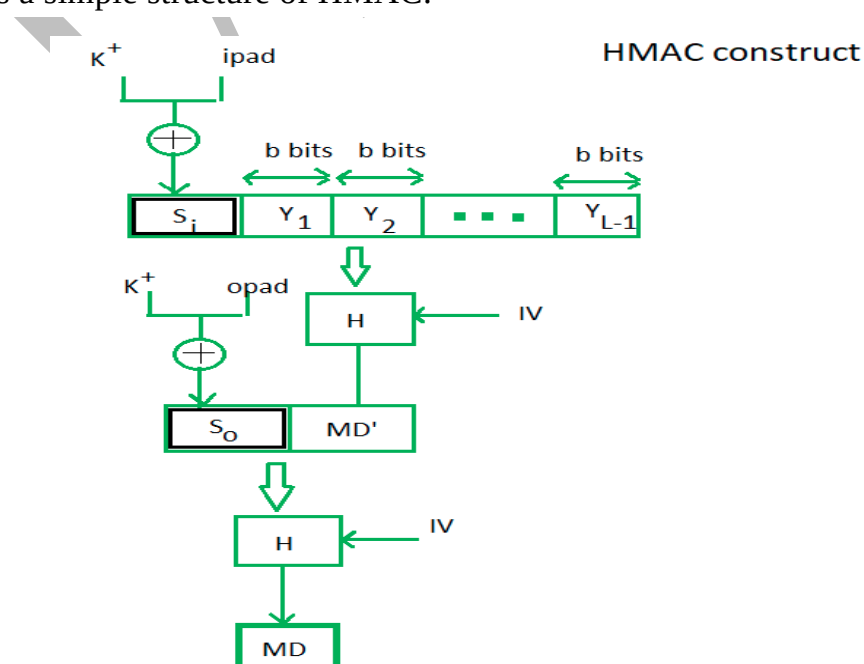
HMAC algorithm stands for Hashed or Hash-based Message Authentication Code. It is a result of work done on developing a MAC derived from cryptographic hash functions. HMAC is a great resistance towards cryptanalysis attacks as it uses the hashing concept twice. HMAC consists of twin benefits of Hashing and MAC and thus is more secure than any other authentication code. RFC 2104 has issued HMAC, and HMAC has been made compulsory to implement in IP security. The FIPS 198 NIST standard has also issued HMAC.

- As the Hash Function, HMAC is also aimed to be one way, i.e, easy to generate output from input but complex the other way round.
- It aims at being less affected by collisions than the hash functions.
- HMAC reuses the algorithms like MD5 and SHA-1 and checks to replace the embedded hash functions with more secure hash functions, in case found.
- HMAC tries to handle the Keys in a simpler manner.

HMAC algorithm –

The working of HMAC starts with taking a message M containing blocks of length b bits. An input signature is padded to the left of the message and the whole is given as input to a hash function which gives us a temporary message-digest MD' . MD' again is appended to an output signature and the whole is applied a hash function again, the result is our final message digest MD .

Here is a simple structure of HMAC:



Here, H stands for hashing function,

M is the original message

S_i and S_o are input and output signatures respectively,

Y_i is the i th block in original message M, where I ranges from $[1, L)$

L = the count of blocks in M

K is the secret key used for hashing

IV is an initial vector (some constant)

The generation of input signature and output signature S_i and S_o respectively.

$$S_i = K^+ \oplus \text{ipad}$$

where K^+ is nothing but K padded with zeros on the left so that the result is b bits in length

$$S_o = K^+ \oplus \text{opad}$$

where ipad and opad are 00110110 and 01011100 respectively taken $b/8$ times repeatedly.

$$MD' = H(S_i || M)$$

$$MD = H(S_o || MD') \quad \text{or} \quad MD = H(S_o || H(S_i || M))$$

To a normal hash function, HMAC adds a compression instance to the processing. This structural implementation holds efficiency for shorter MAC values.



Public Key Cryptography Standards

1. PKCS

In cryptography, PKCS stands for "Public Key Cryptography Standards". These are a group of public-key cryptography standards devised and published by RSA Security LLC, starting in the early 1990s. The company published the standards to promote the use of the cryptography techniques to which they had patents, such as the RSA algorithm, the Schnorr signature algorithm and several others.

Public Key Cryptography Standard (PKCS), which has had a significant impact on the use of public-key encryption in practice. The PKCS standard is a set of standards called PKCS 1 to 15. These standards cover RSA encryption, RSA signature, password-based encryption, encrypted message syntax, private key information syntax, selected object category and attribute type, authentication request syntax, encryption token interface, personal information exchange syntax, and encrypted token information grammar. RSA Laboratories publishes the PKCS standard. Although RSA Laboratories solicits comments and suggestions from the public on the PKCS standard, RSA Laboratories reserves the exclusive power to decide all aspects of the PKCS standard. PKCS has become the basis for many other standards, such as S/MIME.

PKCS did not become industry standards initially because RSA retained control over them, but many of the standards were adapted by other working groups. The standards were developed by RSA with the cooperation of industry partners which included Apple, Microsoft, Lotus, Sun, DEC and MIT.

FIPS 140-2

This standard specifies the security requirements that will be satisfied by a cryptographic module utilized within a security system protecting sensitive but unclassified information (hereafter referred to as sensitive information). The standard provides four increasing, qualitative levels of security: Level 1, Level 2, Level 3, and Level 4. These levels are intended to cover the wide range of potential applications and environments in which cryptographic modules may be employed. The security requirements cover areas related to the secure design and implementation of a cryptographic module. These areas include cryptographic module specification, cryptographic module ports and interfaces; roles, services, and authentication; finite state model; physical security; operational environment; cryptographic key management; electromagnetic interference/electromagnetic compatibility (EMI/EMC); self-tests; design

assurance; and mitigation of other attacks. This standard supersedes FIPS 140-1, Security Requirements for Cryptographic Modules, in its entirety.

The Cryptographic Module Validation Program (CMVP) validates cryptographic modules to Federal Information Processing Standard (FIPS) 140-2 and other cryptography-based standards. The CMVP is a joint effort between NIST and the Communications Security Establishment (CSE) of the Government of Canada. Products validated as conforming to FIPS 140-2 are accepted by the Federal agencies of both countries for the protection of sensitive information (United States) or Designated Information (Canada). The goal of the CMVP is to promote the use of validated cryptographic modules and provide Federal agencies with a security metric to use in procuring equipment containing validated cryptographic modules.

Strong Authentication

Strong Authentication is a method of user verification that is considered robust enough to withstand attacks on the system to which the users are authenticating. There are competing definitions of strong authentication, as layered systems often do not meet the required threshold for being strong. Strong authentication is thought to be true two-factor authentication or multi-factor authentication (2FA, MFA). Such systems, in requiring two or more factors from the “something I have, something I am, and something I know), require those factors to be a combination of different authentication factors. Technologies that enable strong authentication include information derived from the devices in use by people (e.g. mobile or laptop) as well as user biometrics, user email, one-time passwords (OTPs), and time-based one-time passwords (TOTPs).

True strong authentication also takes the form of cryptographic solutions where public-key infrastructure (PKI) is the underlying system. Such systems ensure that the user and the verifying system do not share sensitive information. Rather, the parties exchange non-sensitive mutually agreed upon information to verify that they are the authorized parties to a conversation. Fast Identity Online (FIDO) Alliance authentication standards are examples of such a system.

In addition, in the consumer space, Strong Customer Authentication (SCA) is defined in the recent update to the European Union’s Payment Services Directive (PSD, PSD2). SCA serves as an accepted strong authentication definition.

Single-Factor Authentication (SFA)

Single-Factor Authentication (SFA) is a method of logging users into resources by having them present only one way of verifying their identity. Username and password is the dominant form of SFA.

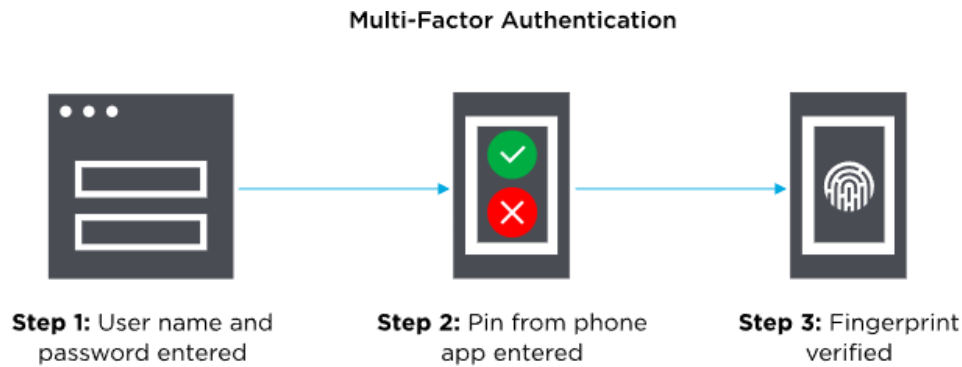
The factors of authenticating users generally take the form of either something you know (knowledge), something you have (possession), and something you are (biometric). Continuous authentication is an emerging means using one factor however even it compiles contextual information derived from knowledge and possession, in addition to device information such as geolocation.

As the number-one SFA method, password authentication relies on mutual secrets between user and the online service. Shared secrets have shown to be vulnerable to credential stuffing attacks since users recycle passwords across different services, and hackers weaponize passwords from prior breaches against unrelated enterprises. In response, enterprises implement two-factor authentication (2FA) or multi-factor authentication (MFA) to add a possession factor often using hardware or software tokens. These methods, however, add friction to the user experience (UX) and result in poor 2FA or MFA adoption, or security workarounds such as sharing tokens.

Everyday mobile devices are helping enterprises overtake the reliance on SFA and the challenges of reliable 2FA and MFA. Smartphones are de-factor MFA and have multiple authenticators, security features such hardware trust zones, and can be leveraged for public-key cryptography (PKC). This makes smartphones crucial to password less authentication based on true secrets, and fast, simple MFA.

Multi-Factor Authentication (MFA)

Multi-factor Authentication (MFA) is an authentication method that requires the user to provide two or more verification factors to gain access to a resource such as an application, online account, or a VPN. MFA is a core component of a strong identity and access management (IAM) policy. Rather than just asking for a username and password, MFA requires one or more additional verification factors, which decreases the likelihood of a successful cyber-attack.



The main benefit of MFA is it will enhance your organization's security by requiring your users to identify themselves by more than a username and password. While important, usernames and passwords are vulnerable to brute force attacks and can be stolen by third parties. Enforcing the use of an MFA factor like a thumbprint or physical hardware key means increased confidence that your organization will stay safe from cyber criminals.

MFA works by requiring additional verification information (factors). One of the most common MFA factors that users encounter is one-time passwords (OTP). OTPs are those 4–8-digit codes that you often receive via email, SMS or some sort of mobile app. With OTPs a new code is generated periodically or each time an authentication request is submitted. The code is generated based upon a seed value that is assigned to the user when they first register and some other factor which could simply be a counter that is incremented or a time value.

Most MFA authentication methodology is based on one of three types of additional information:

- Things you know (knowledge), such as a password or PIN
- Things you have (possession), such as a badge or smartphone
- Things you are (inherence), such as a biometric like fingerprints or voice recognition

Difference between single-factor and multi-factor authentication systems:

Single-factor authentication	Multi-factor authentication
In single-factor authentication, the process of authentication is much simpler	In multi-factor authentication, the process of successful authentication can be complex.
There is a risk of security.	There is no risk of security.
There is a chance of information getting stolen	There is not a chance, for the information to get stolen.
There is a risk of a keylogger who can steal passwords and more.	There is no risk of keylogger activity.
The user is not in full control.	The user is in full control.
There is a chance of user password getting captured by the keylogger.	There is no risk of data getting captured through keylogger.
There is a chance of a phishing attack, where the attacker may deceive the user to enter the password or userid.	In multi-factor authentication, a phishing attack will not accomplish its purpose.

Single Sign-on Solutions

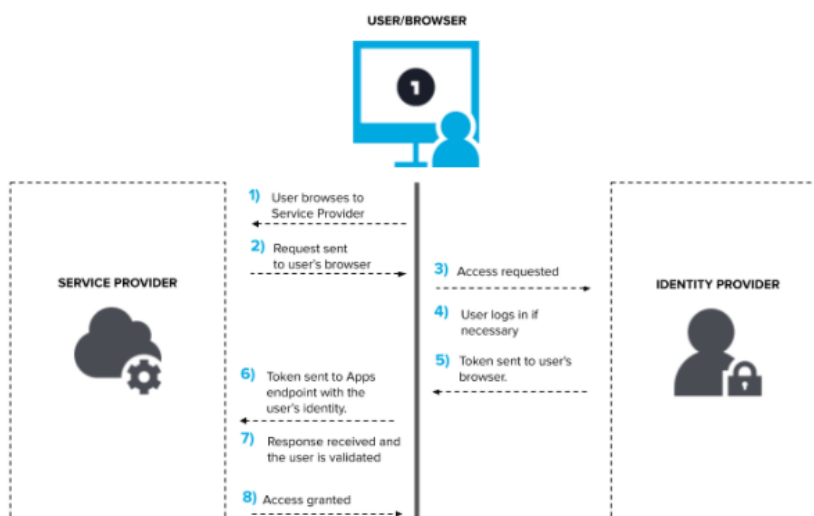
Single sign-on (SSO) is an authentication method that enables users to securely authenticate with multiple applications and websites by using just one set of credentials. SSO works based upon a trust relationship set up between an application, known as the service provider, and an identity provider, like OneLogin. This trust relationship is often based upon a certificate that is exchanged between the identity provider and the service provider. This certificate can be used to sign identity information that is being sent from the identity provider to the service provider so that the service provider knows it is coming from a trusted source. In SSO, this identity data takes the form of

tokens which contain identifying bits of information about the user like a user's email address or a username.

The login flow usually looks like this:

1. A user browses to the application or website they want access to, aka, the Service Provider.
2. The Service Provider sends a token that contains some information about the user, like their email address, to the SSO system, aka, the Identity Provider, as part of a request to authenticate the user.
3. The Identity Provider first checks to see whether the user has already been authenticated, in which case it will grant the user access to the Service Provider application and skip to step 5.
4. If the user hasn't logged in, they will be prompted to do so by providing the credentials required by the Identity Provider. This could simply be a username and password or it might include some other form of authentication like a One-Time Password (OTP).
5. Once the Identity Provider validates the credentials provided, it will send a token back to the Service Provider confirming a successful authentication.
6. This token is passed through the user's browser to the Service Provider.
7. The token that is received by the Service Provider is validated according to the trust relationship that was set up between the Service Provider and the Identity Provider during the initial configuration.
8. The user is granted access to the Service Provider.

Service Provider Initiated Workflow



When the user tries to access a different website, the new website would have to have a similar trust relationship configured with the SSO solution and the authentication flow would follow the same steps.

An SSO token is a collection of data or information that is passed from one system to another during the SSO process. The data can simply be a user's email address and information about which system is sending the token. Tokens must be digitally signed for the token receiver to verify that the token is coming from a trusted source. The certificate that is used for this digital signature is exchanged during the initial configuration process.



There are many reasons why SSO can improve security. A single sign-on solution can simplify username and password management for both users and administrators. Users no longer have to keep track of different sets of credentials and can simply remember a single more complex password. SSO often enables users to just get access to their applications much faster.

SSO can also cut down on the amount of time the help desk has to spend on assisting users with lost passwords. Administrators can centrally control requirements like password complexity and multi-factor authentication (MFA). Administrators can also more quickly relinquish login privileges across the board when a user leaves the organization.

Single Sign-On does have some drawbacks. For example, you might have applications that you want to have locked down a bit more. For this reason, it would be important to choose an SSO solution that gives you the ability to, say, require an additional authentication factor before a user logs into a particular application or that prevents users from accessing certain applications unless they are connected to a secure network.



OpenID

OpenID allows you to use an existing account to sign in to multiple websites, without needing to create new passwords.

You may choose to associate information with your OpenID that can be shared with the websites you visit, such as a name or email address. With OpenID, you control how much of that information is shared with the websites you visit.

With OpenID, your password is only given to your identity provider, and that provider then confirms your identity to the websites you visit. Other than your

provider, no website ever sees your password, so you don't need to worry about an unscrupulous or insecure website compromising your identity.

OpenID is rapidly gaining adoption on the web, with over one billion OpenID enabled user accounts and over 50,000 websites accepting OpenID for logins. Several large organizations either issue or accept OpenIDs, including Google, Facebook, Yahoo!, Microsoft, AOL, MySpace, Sears, Universal Music Group, France Telecom, Novell, Sun, Telecom Italia, and many more.

OAuth

OAuth is an open-standard authorization protocol or framework that provides applications the ability for "secure designated access." For example, you can tell Facebook that it's OK for ESPN.com to access your profile or post updates to your timeline without having to give ESPN your Facebook password. This minimizes risk in a major way: In the event ESPN suffers a breach, your Facebook password remains safe.

OAuth doesn't share password data but instead uses authorization tokens to prove an identity between consumers and service providers. OAuth is an authentication protocol that allows you to approve one application interacting with another on your behalf without giving away your password.

NOTE: OpenID is used for authentication while OAuth is used for authorization.

Graphical Passwords

A graphical password or graphical user authentication is a form of authentication using images rather than letters, digits, or special characters. The type of images used and the ways in which users interact with them vary between implementations.

With increasing technical advancements, the world is becoming digital at a high pace and everything is happening online. From paying your bills to ticket bookings to paying the person sitting next to you, you prefer to pay online. Not only payments but all activities, be it, communication through e-mails and messaging apps, keeping your documents in a digital locker, etc happen online.

With everything turning online, the risk of cybercrimes and privacy breaches is also increasing. Passwords play a huge role in keeping your data safe online as well as offline platforms. Passwords are the default method of authentication

to get access to our accounts. There are various types of authentications available for users to secure their accounts.

Types of authentications

- **Token-based authentication** includes key cards, bank cards, smart cards, etc.
- **Knowledge-based authentication** includes text-based authentication and picture-based authentication.
- **Biometric authentication** includes fingerprints authentication, iris scan and facial recognition.

Considering the traditional username-password authentication, the alphanumeric passwords are either easy to guess or difficult to remember. Also, users generally keep the same passwords for all their accounts because it is difficult to remember a lot of them. Alternative authentication methods, such as biometrics, graphical passwords are used to overcome these problems associated with the traditional username-password authentication technique.

In a graphical password authentication system, the user has to select from images, in a specific order, presented to them in a graphical user interface (GUI). According to a study, the human brain has a greater capability of remembering what they see (pictures) rather than alphanumeric characters. Therefore, graphical passwords overcome the disadvantage of alphanumeric passwords. Graphical Password Authentication has three major categories based on the activity they use for authentication of the password:

- **Recognition based Authentication:** A user is given a set of images and he has to identify the image he selected during registration. For example, *Passfaces* is a graphical password scheme based on recognizing human faces. During password creation, users are given a large set of images to select from. To log in, users have to identify the pre-selected image from the several images presented to him.
- **Recall based Authentication:** A user is asked to reproduce something that he created or selected at the registration stage. For example, in the *Passpoint* scheme, a user can click any point in an image to create the password and a tolerance around each pixel is calculated. During authentication, the user has to select the points within the tolerance in the correct sequence to login.
- **Cued Recall:** Cued Click Points (CCP) is an alternative to the PassPoints technique. In CCP, users click one point on each image rather

than on five points on one image (unlike PassPoints). It offers cued-recall and instantly alerts the users if they make a mistake while entering their latest click-point.

Advantages:

- It is user-friendly.
- It provides higher security than other traditional password schemes.
- Dictionary attacks are infeasible.
- CCP makes attacks based on hotspot analysis more challenging.

Disadvantages:

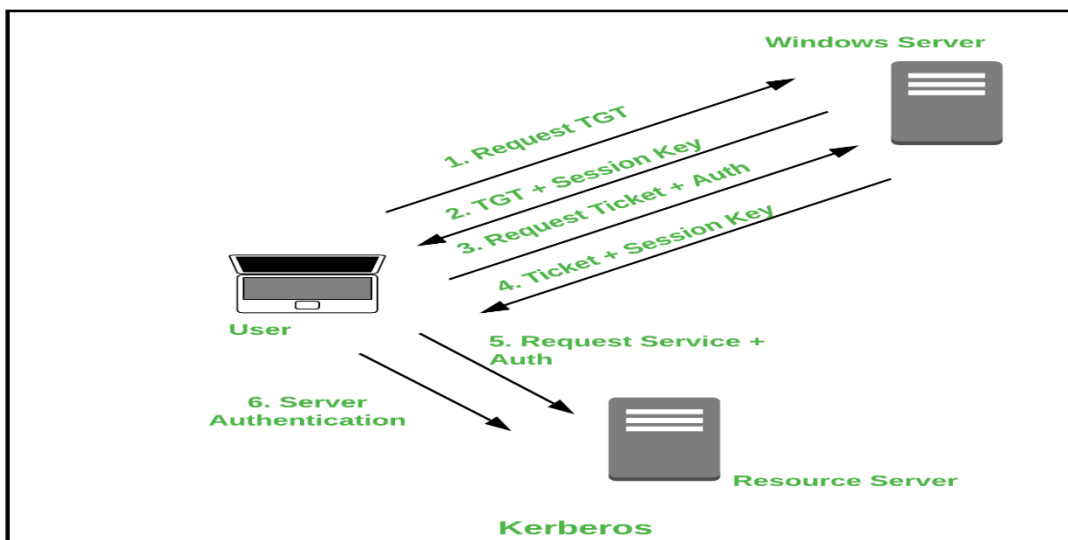
- Registration and login take too long.
- It requires more storage space because of images.
- Shoulder surfing (Watching over people's shoulders as they process information).

Authentication Protocols

User authentication is the first most priority while responding to the request made by the user to the software application. There are several mechanisms made which are required to authenticate the access while providing access to the data. In this blog, we will explore the most common authentication protocols and will try to explore their merits and demerits.

1. Kerberos :

Kerberos is a protocol that aids in network authentication. This is used for validating clients/servers during a network employing a cryptographic key. It is designed for executing strong authentication while reporting to applications. The overall implementation of the Kerberos protocol is openly available by MIT and is used in many mass-produced products.



Some advantages of Kerberos:

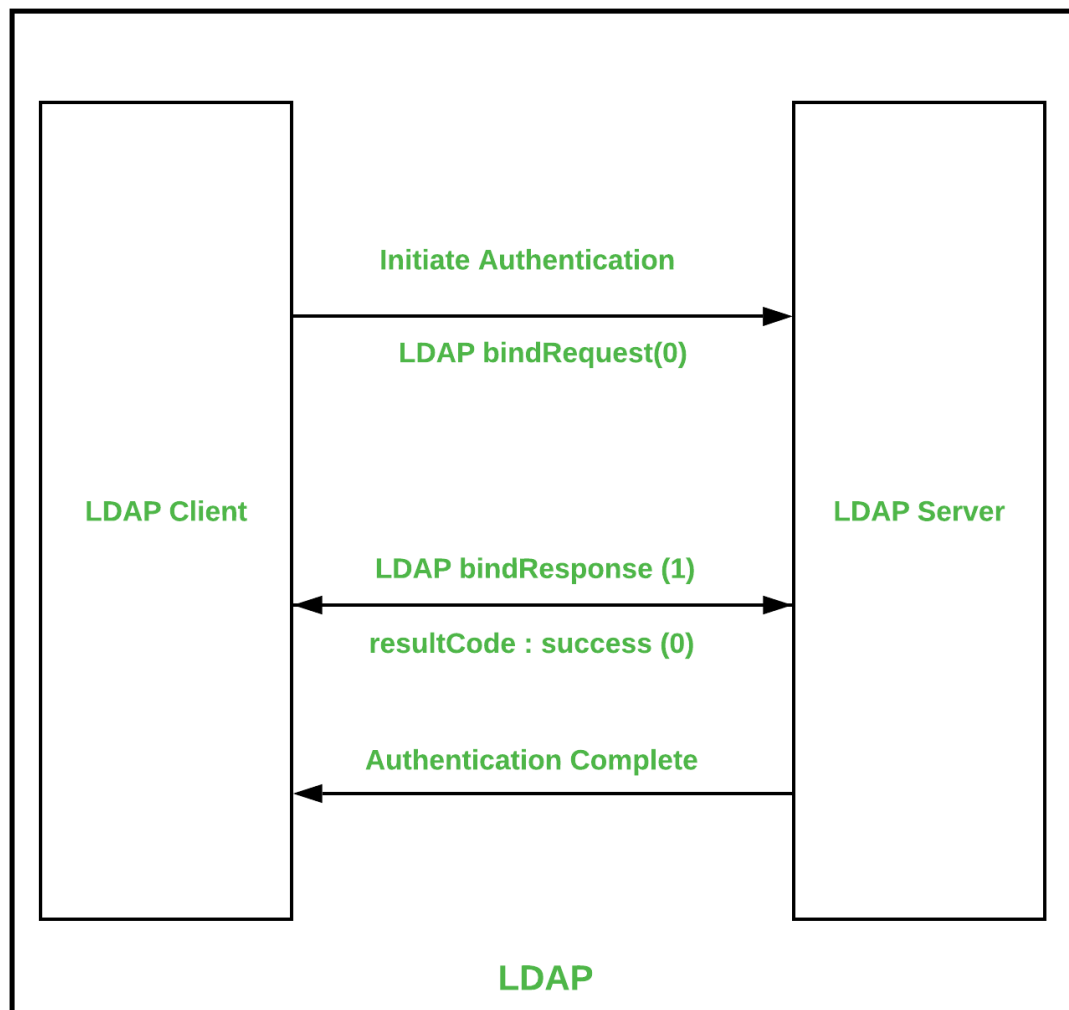
- It supports various operating systems.
- The authentication key is shared much efficiently than public sharing.

Some disadvantages of Kerberos:

- It is used only to authenticate clients and services used by them.
- It shows vulnerability to soft or weak passwords.

2. Lightweight Directory Access Protocol (LDAP):

LDAP refers to Lightweight Directory Access Protocol. It is a protocol that is used for determining any individuals, organizations, and other devices during a network regardless of being on public or corporate internet. It is practiced as Directories-as-a-Service and is the grounds for Microsoft building Activity Directory.



Some advantages of LDAP:

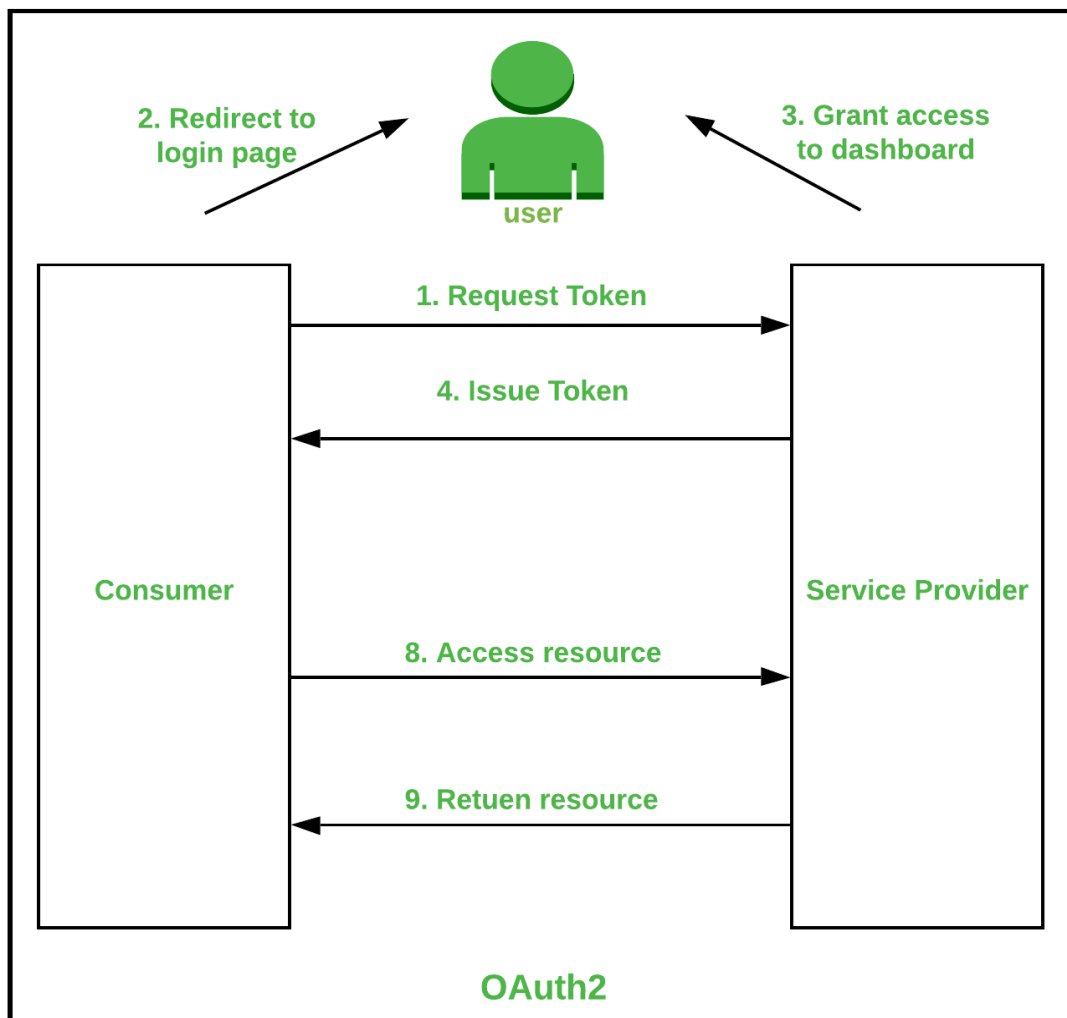
- It is an automated protocol which makes it modernizing easier.
- It supports existing technologies and allows multiple directories.

Some disadvantages of LDAP:

- It requires the experience of deployment.
- The directory servers are required to be LDAP obedient for deployment.
-

3. OAuth2 :

OAuth as the name suggests it is an authorization framework that promotes granting limited access to the user on its account through an HTTP service. When a user requests access to resources an API call is made and after the authentication token is passed.

**Some advantages of OAuth2:**

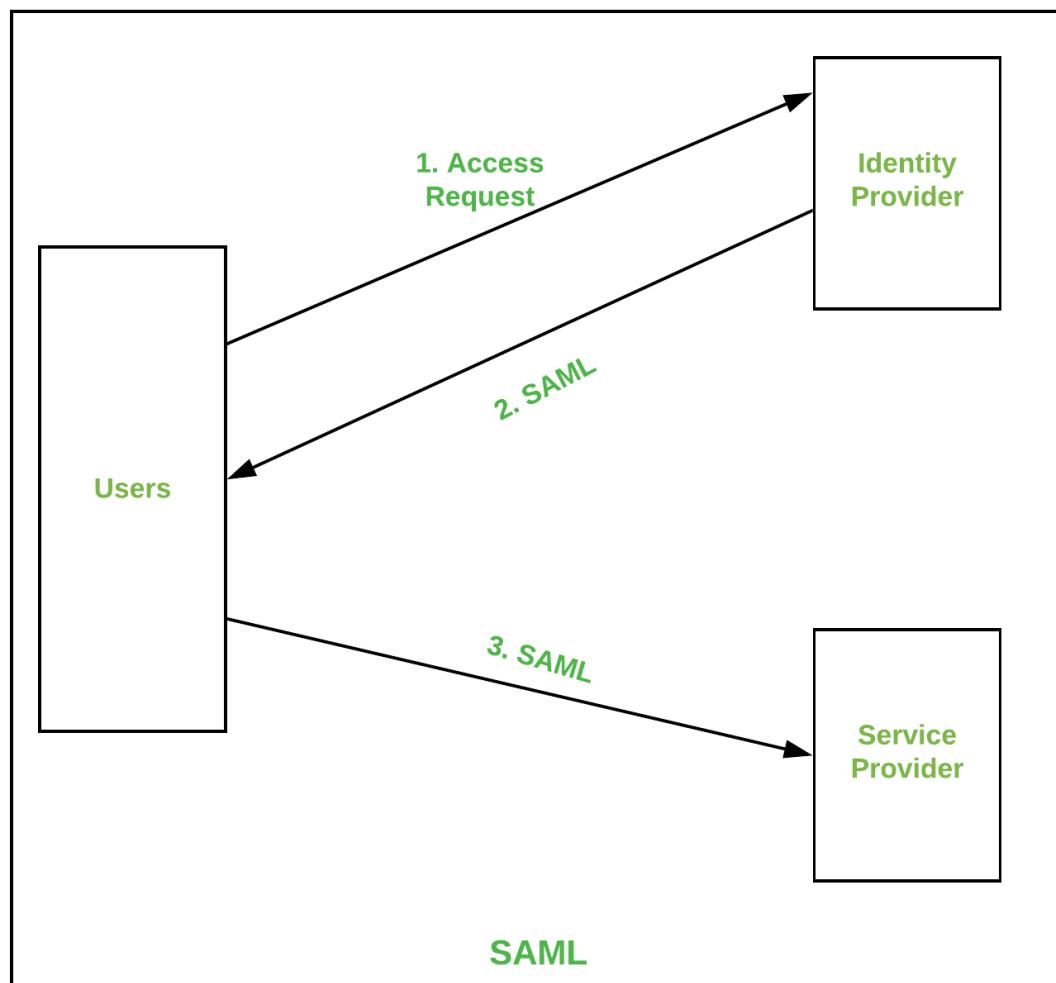
- It is a simple protocol and is easy to implement.
- It provides server-side authorization of code.

Some disadvantages of OAuth2:

- It is vulnerable to manage different sets of code.
- It shows serious effects on sites connected to another affected system.
-

4. SAML :

SAML stands for Security Assertion Markup Language which is based on XML-based authentication data format which provides the authorization between an identity provider and service provider. It serves as a product of the OASIS Security Services Technical Committee.

**Some advantages of SAML:**

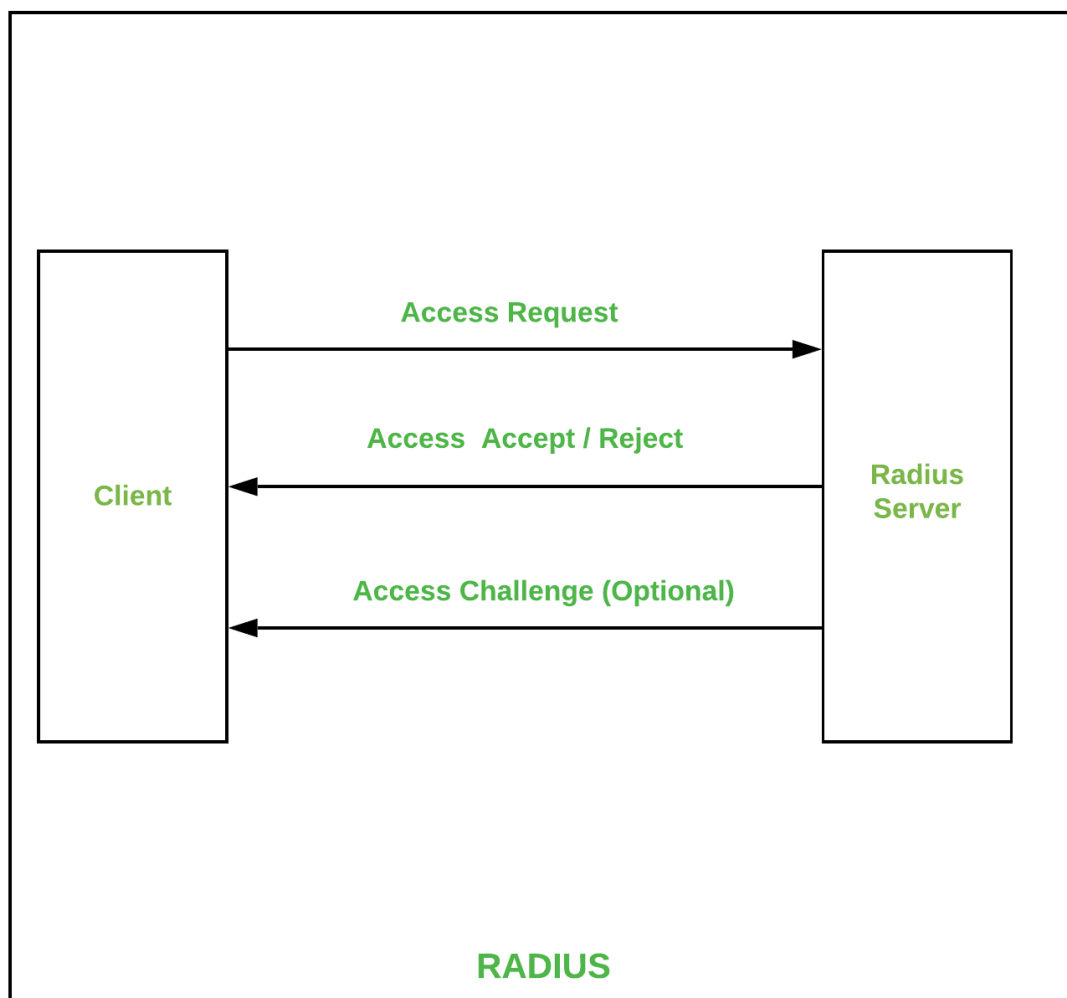
- It reduced the administrative costs for the end-users.
- It provides a single sign-in for authenticating across service providers.

Some disadvantages of SAML:

- It is dependent on the identity provider.
- All the data is managed in a single XML format.

5. RADIUS :

RADIUS stands for Remote Authentication Dial-In User Service. It is a network protocol that provides sufficient centralized Authentication, Accounting, and Authorization for the users that use and network services. The functioning of the protocol occurs when the user requests access to network resources, where the RADIUS server encrypts the credentials which are entered by the user. After this, the user credentials are mapped through the local database and provide access.



Some advantages of RADIUS:

- It is a great mechanism for providing multiple access for Admins.
- It provides a unique identity to each user in a session.

Some disadvantages of RADIUS:

- Initial implementation for this mechanism is hard on hardware.
- It has a variety of models that may require a special team which is cost consuming.

FIDO

Fast Identity Online (FIDO) Authentication is a set of open technical specifications that define user authentication mechanisms that reduce the reliance on passwords.

FIDO protocols are designed from the ground up to protect user privacy. The protocols do not disclose sensitive user data that can be used by different online services to collaborate and track a user across the services. Other sensitive data like biometric prints and PINs never leaves the user's device to ensure it cannot be intercepted or compromised by an attacker.

To authenticate a user, an application – often referred to as the relying party – uses FIDO-specified client-side APIs to interact with a user's registered authenticator. For web applications, client-side APIs include WebAuthn implemented by the web browser, which in turn calls on FIDO CTAP to access the authenticator.

To authenticate a user, the relying party passes a cryptographic challenge to the registered authenticator and evaluates the response to determine the authenticity of the secrets stored on the client device and used to produce the response.

“Under the hood” FIDO utilizes asymmetric cryptography to ensure that all sensitive secrets and cryptographic key material remain on the client device at all times and are not transmitted to the authenticating service.

FIDO authentication requires an initial registration step. In cases where the user device supports multiple forms of authentication (i.e. fingerprint scanner, voiceprint recorder, face ID, etc.), the user is asked to choose a FIDO compliant authenticator from the options available on the device that matches the authenticating app's acceptance policy. The user then unlocks the FIDO authenticator using whatever mechanism is built into the authenticator – e.g. by providing a fingerprint, pressing a button on a second-factor device, or entering PIN.

Once the authenticator is unlocked, the user's device creates a new and unique public/private cryptographic key pair that will be used for authenticating access. The public key is then sent to the online service and associated with the user's account. The private key and all other sensitive data related to the chosen authentication method – for example, biometric prints – remain on the local device and never leave it.

Authentication requires the client device to prove possession of the private key to the authenticating service by successfully responding to a cryptographic challenge. The private key can only be used after successfully authenticating using the registered authenticator, for example by swiping a finger on the fingerprint sensor, entering a PIN, speaking into a microphone, inserting a second-factor device, pressing a button, etc. The device then uses the user account identifier provided by the service to select the correct key and cryptographically sign the service's challenge. The signed challenge is sent back to the service, which verifies it with the stored public key and logs in the user.

Zero Trust Architecture

Security model that requires strict verification for every person & device trying to access resources on a private network regardless of whether they are sitting within or outside the network perimeter.

It is also known as Zero trust security or Zero trust network.

No one trusted by default.



➤ Identity Access Management

Identity and access management (IAM) ensures that the right people and job roles in your organization (identities) can access the tools they need to do their jobs. Identity management and access systems enable your organization to manage employee apps without logging into each app as an administrator. Identity and access management systems enable your organization to manage a range of identities including people, software, and hardware like robotics and IoT devices.

Companies need IAM to provide online security and to increase employee productivity.

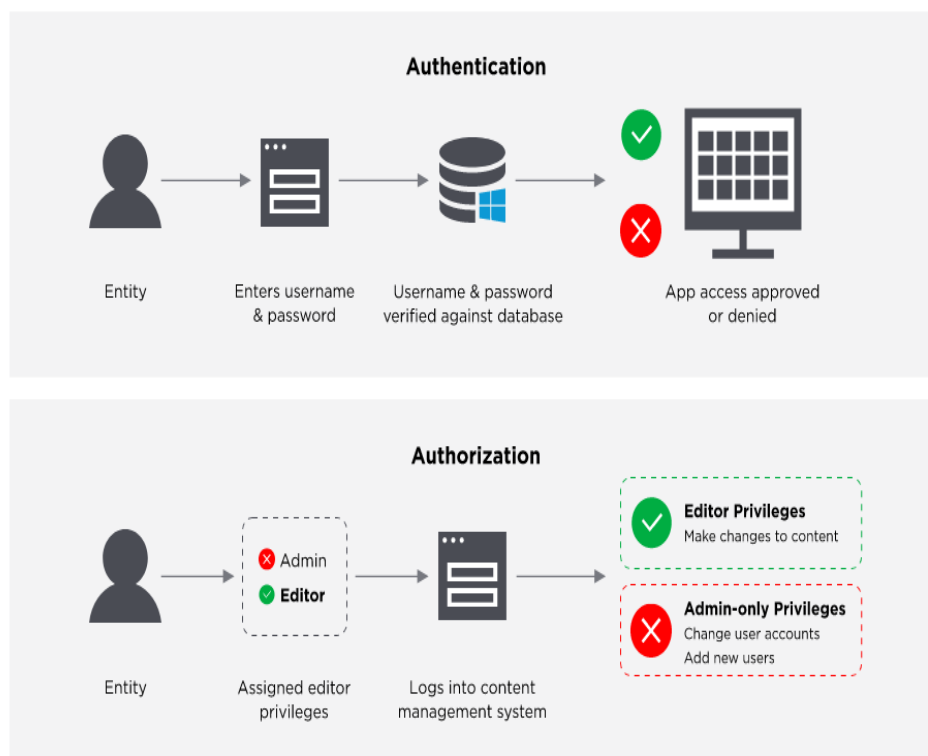
- **Security.** Traditional security often has one point of failure - the password. If a user's password is breached - or worse yet, the email address for their password recoveries - your organization becomes vulnerable to attack. IAM services

narrow the points of failure and backstop them with tools to catch mistakes when they're made.

- **Productivity.** Once you log on to your main IAM portal, your employee no longer has to worry about having the right password or right access level to perform their duties. Not only does every employee get access to the perfect suite of tools for their job, their access can be managed as a group or role instead of individually, reducing the workload on your IT professionals.

Identity management solutions generally perform two tasks:

1. IAM confirms that the user, software, or hardware is who they say they are by authenticating their credentials against a database. IAM cloud identity tools are more secure and flexible than traditional username and password solutions.
2. Identity access management systems grant only the appropriate level of access. Instead of a username and password allowing access to an entire software suite, IAM allows for narrow slices of access to be portioned out, i.e. *editor*, *viewer*, and *commenter* in a content management system.





Privileged Access Management

Privileged Access Management (PAM) is an information security (infosec) mechanism that safeguards identities with special access or capabilities beyond regular users. Like all other infosec solutions, PAM works through a combination of people, processes, and technology.

We treat privileged accounts with extra care because of the risk they pose to the technology environment. For example, should the credentials of an administrator or service account fall into the wrong hands, it could lead to the compromise of the organization's systems and confidential data.

Data breaches occur when threat actors compromise privileged access accounts. As these accounts hold the keys that unlock every door in a technology environment, we need to add additional layers of protection. That extra security is a Privileged Access Management solution.

What does Privileged Access mean?

In a technology environment, privileged access refers to accounts with elevated capabilities beyond regular users. For example, in a Linux environment, the root user can add, amend, or delete users, install and uninstall software, and access restricted parts of the operating system that are off-limits to a standard user. Windows environments follow a similar security construct, but the root user in that instance is called an administrator.

Let's illustrate the concept of privileged access with a real-world banking example. A typical bank has customers, tellers, and managers. Each 'user' has different levels of authority when it comes to accessing the bank's cash. Customers can only access the money in their bank accounts. Tellers have more privileges than regular customers as they have access to all the cash in their respective drawers. Managers have even greater access than tellers, as they can access the money stored in the bank's vault. Technology systems also use this tiered privilege access model. Your role within the system determines what you can or cannot do.

In our banking example, the tellers and managers would be the users with privileged access. As these roles have access to more of the bank's cash than customers, the bank needs to implement additional security measures before granting tellers and manager's access. For instance, during their job interviews, they may need to pass a criminal record check. When they start

working at the bank, their role will also determine their physical access. For example, tellers may be able to enter the secure area of the bank, but only managers will have the privileged access needed to enter the vault.

PAM vs. IAM

Privileged Access Management (PAM) is a component of a broader Identity and Access Management (IAM) solution. A PAM deals with the process and technologies needed to secure privileged accounts. On the other hand, an IAM solution offers password management, Multi-Factor Authentication, Single Sign-On (SSO), and user lifecycle management for all accounts, not just those with privileged access.

Securing Websites & Email

➤ SSL

Secure Socket Layer (SSL) is a technology to secure the communication between a client and the server. An SSL for email ensures that the email is not intercepted during the transit and nobody except the intended recipient can access it. An email SSL certificate can also authenticate the identity of the sender. The SSL certificate in your email account serves two purposes – to authenticate the sender's identity and maintain the integrity of the email. The email certificates are also known as S/MIME or email encryption certificates.

➤ TLS

Transport Layer Securities (TLS) are designed to provide security at the transport layer. TLS was derived from a security protocol called Secure Service Layer (SSL). TLS ensures that no third party may eavesdrops or tampers with any message.

There are several benefits of TLS:

- **Encryption:**
TLS/SSL can help to secure transmitted data using encryption.
- **Interoperability:**
TLS/SSL works with most web browsers, including Microsoft Internet Explorer and on most operating systems and web servers.

- Algorithm flexibility:
TLS/SSL provides operations for authentication mechanism, encryption algorithms and hashing algorithm that are used during the secure session.
- Ease of Deployment:
Many applications TLS/SSL temporarily on a windows server 2003 operating systems.
- Ease of Use:
Because we implement TLS/SSL beneath the application layer, most of its operations are completely invisible to client.

Working of TLS:

The client connects to server (using TCP), the client will be something. The client sends number of specifications:

1. Version of SSL/TLS.
2. which cipher suites, compression method it wants to use.

The server checks what the highest SSL/TLS version is that is supported by them both, picks a cipher suite from one of the client's options (if it supports one) and optionally picks a compression method. After this the basic setup is done, the server provides its certificate. This certificate must be trusted either by the client itself or a party that the client trusts. Having verified the certificate and being certain this server really is who he claims to be (and not a man in the middle), a key is exchanged. This can be a public key, "PreMasterSecret" or simply nothing depending upon cipher suite.

Both the server and client can now compute the key for symmetric encryption. The handshake is finished and the two hosts can communicate securely. To close a connection by finishing. TCP connection both sides will know the connection was improperly terminated. The connection cannot be compromised by this through, merely interrupted.

➤ **PGP – Authentication and Confidentiality**

In 2013, when the *NSA (United States National Security Agency) scandal* was leaked to the public, people started to opt for the services which can provide

them a strong privacy for their data. Among the services people opted for, most particularly for emails, were different plug-ins and extensions for their browsers. Interestingly, among the various plug-ins and extensions that people started to use, there were two main programs that were solely responsible for the complete email security that the people needed.

As said, PGP (Pretty Good Privacy), is a popular program that is used to provide confidentiality and authentication services for electronic mail and file storage. It was designed by Phil Zimmermann way back in 1991. He designed it in such a way, that the best cryptographic algorithms such as RSA, Diffie-Hellman key exchange, DSS are used for the public-key encryption (or) asymmetric encryption; CAST-128, 3DES, IDEA are used for symmetric encryption and SHA-1 is used for hashing purposes. PGP software is an open source one and is not dependent on either of the OS (Operating System) or the processor. The application is based on a few commands which are very easy to use.

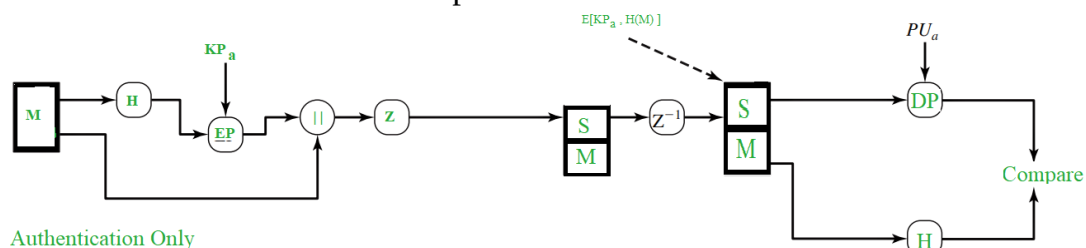
The following are the services offered by PGP:

1. Authentication
2. Confidentiality
3. Compression
4. Email Compatibility
5. Segmentation

1. Authentication:

Authentication basically means something that is used to validate something as true or real. To login into some sites sometimes we give our account name and password, that is an authentication verification procedure.

In the email world, checking the authenticity of an email is nothing but to check *whether it actually came from the person it says*. In emails, authentication has to be checked as there are some people who spoof the emails or some spams and sometimes it can cause a lot of inconvenience. The Authentication service in PGP is provided as follows:



As shown in the above figure, the Hash Function (H) calculates the Hash Value of the message. For the hashing purpose, SHA-1 is used and it produces a 160 bit output hash value. Then, using the sender's private key (KP_a), it is encrypted and it's called as Digital Signature. The Message is then appended to the signature. All the process happened till now, is sometimes described as *signing the message*. Then the message is compressed to reduce the transmission overhead and is sent over to the receiver.

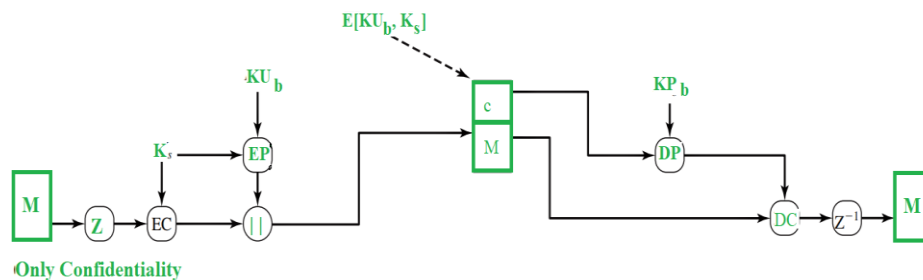
At the receiver's end, the data is decompressed and the message, signature are obtained. The signature is then decrypted using the sender's public key (PU_a) and the hash value is obtained. The message is again passed to hash function and it's hash value is calculated and obtained.

Both the values, one from signature and another from the recent output of hash function are compared and if both are same, it means that the email is actually sent from a known one and is legit, else it means that it's not a legit one.

2. Confidentiality:

Sometimes we see some packages labelled as 'Confidential', which means that those packages are not meant for all the people and only selected persons can see them. The same applies to the email confidentiality as well. Here, in the email service, only the sender and the receiver should be able to read the message, that means the contents have to be kept secret from every other person, except for those two.

PGP provides that Confidentiality service in the following manner:



The message is first compressed and a 128 bit session key (K_s), generated by the PGP, is used to encrypt the message through symmetric encryption. Then, the session key (K_s) itself gets encrypted through public key encryption (EP) using receiver's public key (KU_b). Both the encrypted entities are now concatenated and sent to the receiver.

As you can see, the original message was compressed and then encrypted initially and hence even if any one could get hold of the traffic, he cannot read the contents as they are not in readable form and they can only read them if they had the session key (K_s). Even though session key is transmitted to the receiver and hence, is in the traffic, it is in encrypted form and only the

receiver's private key (K_{P_b}) can be used to decrypt that and thus our message would be completely safe.

At the receiver's end, the encrypted session key is decrypted using receiver's private key (K_{P_b}) and the message is decrypted with the obtained session key. Then, the message is decompressed to obtain the original message (M).

RSA algorithm is used for the public-key encryption and for the symmetric key encryption, CAST-128 (or IDEA or 3DES) is used.

➤ **Secure/Multipurpose Internet Mail Extension (S/MIME):**

S/MIME is a security-enhanced version of Multipurpose Internet Mail Extension (MIME). In this, public key cryptography is used for digital sign, encrypt or decrypt the email. User acquires a public-private key pair with a trusted authority and then makes appropriate use of those keys with email applications.

Difference between PGP and S/MIME

S.NO	PGP	S/MIME
1.	It is designed for processing the plain texts	While it is designed to process email as well as many multimedia files.
2.	PGP is less costly as compared to S/MIME.	While S/MIME is comparatively expensive.
3.	PGP is good for personal as well as office use.	While it is good for industrial use.
4.	PGP is less efficient than S/MIME.	While it is more efficient than PGP.
5.	It depends on user key exchange.	Whereas it relies on a hierarchically valid certificate for key exchange.

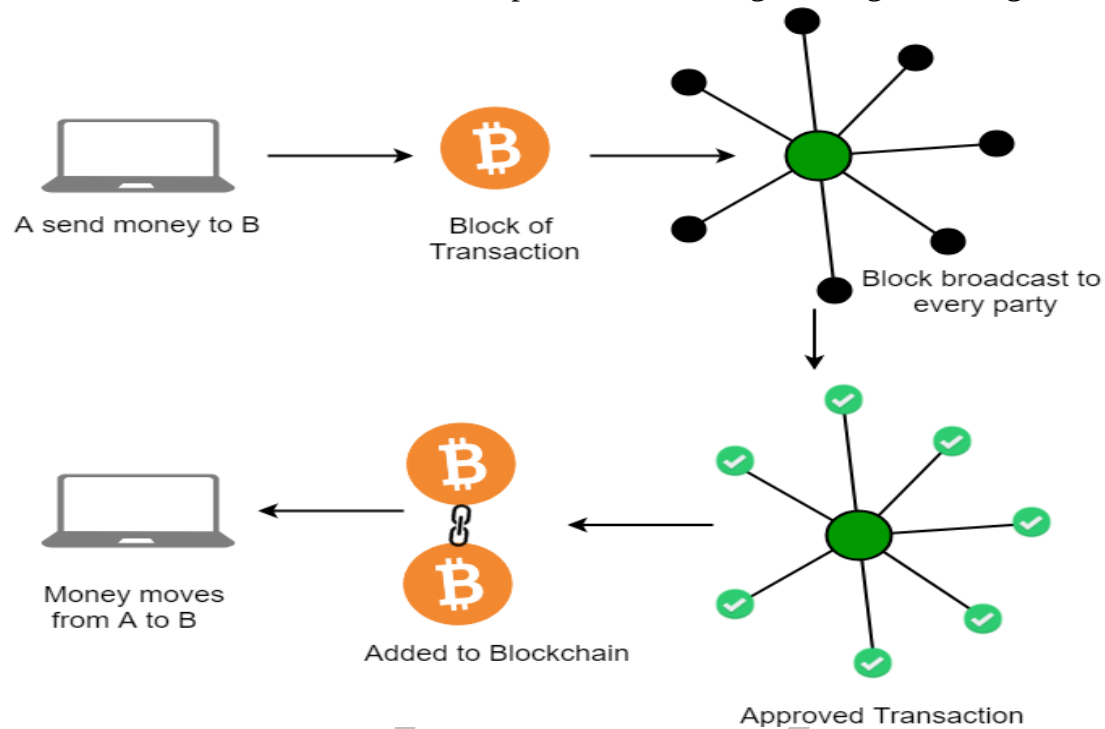
S.NO	PGP	S/MIME
6.	PGP is comparatively less convenient.	While it is more convenient than PGP due to the secure transformation of all the applications.
7.	PGP contains 4096 public keys.	While it contains only 1024 public keys.
8.	PGP is the standard for strong encryption.	While it is also the standard for strong encryption but has some drawbacks.
9.	PGP is also be used in VPNs.	While it is not used in VPNs, it is only used in email services.
10.	PGP uses Diffie hellman digital signature.	While it uses Elgamal digital signature.

Introduction to Blockchain technology

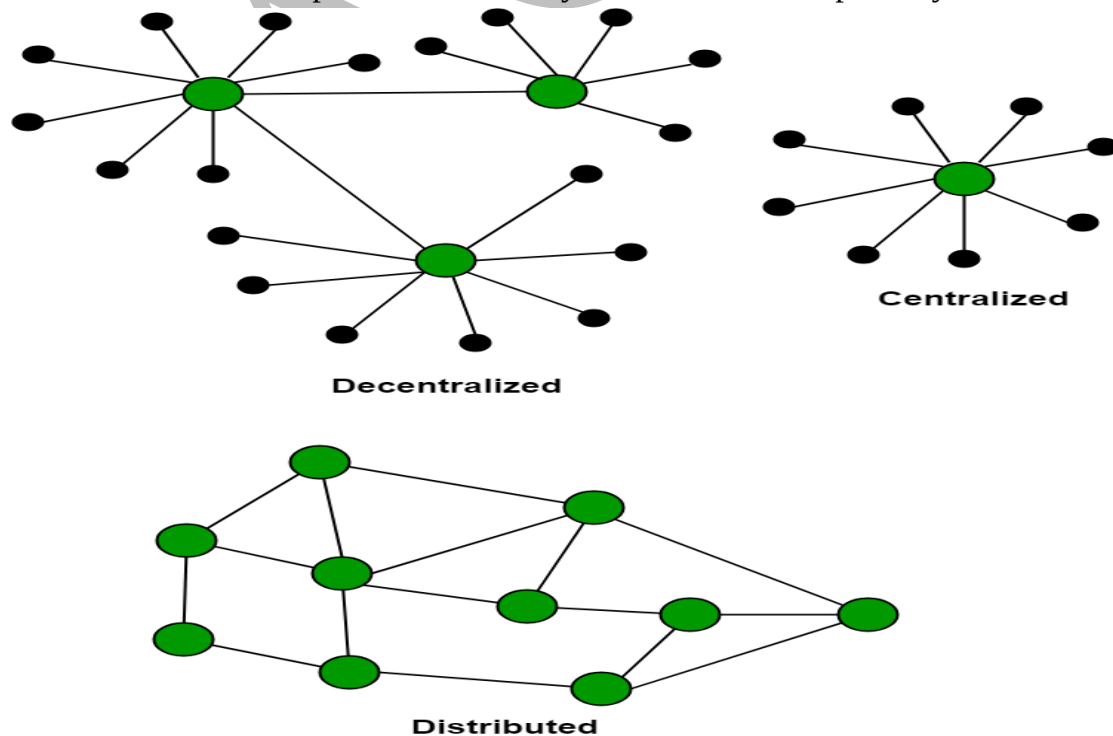
Blockchain is the backbone Technology of Digital Cryptocurrency Bitcoin. The blockchain is a distributed database of records of all transactions or digital event that have been executed and shared among participating parties. Each transaction verified by the majority of participants of the system. It contains every single record of each transaction. Bitcoin is the most popular cryptocurrency an example of the blockchain. Blockchain Technology first came to light when a person or Group of individuals name 'Satoshi Nakamoto' published a white paper on "*Bitcoin: A peer to peer electronic cash system*" in 2008. Blockchain Technology Records Transaction in Digital Ledger which is distributed over the Network thus making it incorruptible. Anything of value like Land Assets, Cars, etc. can be recorded on Blockchain as a Transaction.

How Blockchain Technology works?
One of the famous uses of Blockchain is Bitcoin. The bitcoin is a cryptocurrency

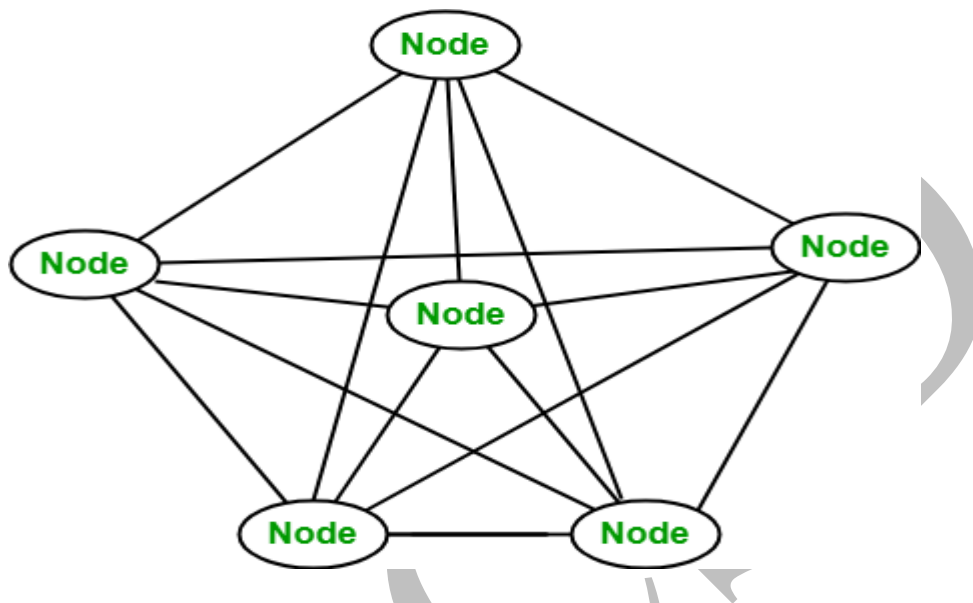
and is used to exchange digital assets online. Bitcoin uses cryptographic proof instead of third-party trust for two parties to execute transactions over the internet. Each transaction protects through digital signature.



Distributed Database: There is no Central Server or System which keeps the data of Blockchain. The data is distributed over Millions of Computers around the world which are connected with the Blockchain. This system allows Notarization of Data as it is present on every Node and is publicly verifiable.



A network of nodes: A node is a computer connected to the Blockchain Network. Node gets connected with Blockchain using the client. Client helps in validating and propagates transaction on to the Blockchain. When a computer connects to the Blockchain, a copy of the Blockchain data gets downloaded into the system and the node comes in sync with the latest block of data on Blockchain. The Node connected to the Blockchain which helps in the execution of a Transaction in return for an incentive is called Miners.



Disadvantages of current transaction system:

- Cash can only be used in low amount transaction locally.
- Huge waiting time in the processing of transactions.
- Need to third party for verification and execution of Transaction make the process complex.
- If the Central Server like Banks is compromised, whole System is affected including the participants.
- Organization doing validation charge high process thus making the process expensive.

Building trust with Blockchain:
Blockchain enhances trust across a business network. It's not that you can't trust those who you conduct business with its that you don't need to when operating on a Blockchain network.

Blockchain built trust through the following five attributes:

- **Distributed:** The distributed ledger is shared and updated with every incoming transaction among the nodes connected to the Blockchain. All this is done in real-time as there is no central server controlling the data.
- **Secure:** There is no unauthorized access to Blockchain made possible through Permissions and Cryptography.
- **Transparent:** Because every node or participant in Blockchain has a copy of the Blockchain data, they have access to all transaction data. They themselves can verify the identities without the need for mediators.
- **Consensus-based:** All relevant network participants must agree that a transaction is valid. This is achieved through the use of consensus algorithms.
- **Flexible:** Smart Contracts which are executed based on certain conditions can be written into the platform. Blockchain Network can evolve in pace with business processes.

Benefits of Blockchain Technology:

- **Time-saving:** No central Authority verification needed for settlements making the process faster and cheaper.
- **Cost-saving:** A Blockchain network reduces expenses in several ways. No need for third-party verification. Participants can share assets directly. Intermediaries are reduced. Transaction efforts are minimized as every participant has a copy of shared ledger.
- **Tighter security:** No one can temper with Blockchain Data as it shared among millions of Participants. The system is safe against cybercrimes and Fraud.